1 El problema

Con el código base (rama 05-mensajeria-base), la transferencia es una saga: cargo, abono y asientos. Al final, cuando el dinero ya llegó a destino, la saga llama por HTTP a notificación y espera su respuesta antes de marcar la transferencia como COMPLETADA. Para ver el problema apagué notificación e hice una transferencia de 50 de Ana a Beto.

1.1 Parte 0: transferencia con notificación apagado

Respuesta de la transferencia de 50 con notificación apagado

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-S9/mexi-banco-s09 (trabajo-s09)
$ curl -i -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' \
  -d '{"claveOrigen":"00218000000000001","claveDestino":"00218000000000002","monto":50}'
HTTP/1.1 500 Internal Server Error
transfer-encoding: chunked
Content-Type: application/json
Date: Mon, 19 Oct 2026 23:33:18 GMT

{"timestamp":"2026-10-19T23:33:18.431+00:00","status":500,"error":"Internal Server Error","path":"/transferencias"}
```

La transferencia quedó en estado ABONADA

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-S9/mexi-banco-s09 (trabajo-s09)
$ curl -s 'http://localhost:8080/api/transferencias?estado=ABONADA'
[{"id":7,"claveOrigen":"00218000000000001","claveDestino":"00218000000000002","monto":50.00,"fecha":"2026-10-19T23:33:18.046603Z","estad
o":"ABONADA","motivo":null,"saldoOrigenTrasCargo":950.00,"intentosDeCompensacion":0,"actualizada":"2026-10-19T23:33:18.249494Z"}]
```

Saldos de Ana y Beto después del error

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-S9/mexi-banco-s09 (trabajo-s09)
$ curl -s http://localhost:8080/api/cuentas/00218000000000001
{"id":26,"clabe":"00218000000000001","titular":"Ana","saldo":950.00}
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-S9/mexi-banco-s09 (trabajo-s09)
$ curl -s http://localhost:8080/api/cuentas/00218000000000002
{"id":27,"clabe":"00218000000000002","titular":"Beto","saldo":550.00}
```

El cliente recibió un error, pero Ana quedó con 950 y Beto con 550: el cargo y el abono sí se aplicaron. La transferencia quedó en ABONADA y siguió así después de volver a encender notificación, porque nadie retoma la saga en ese punto.

1.2 ¿Por qué pasó?

Porque la saga seguía esperando la respuesta HTTP de notificación antes de marcar la transferencia como COMPLETADA, y como notificación no contestó, la excepción salió de notificar y llegó al cliente como error, aunque el dinero ya se había movido.

2 Publicar el evento

Cambié la llamada HTTP por un evento: cuando la saga termina, transferencia publica un mensaje en el exchange `transferencias` de RabbitMQ y sigue con lo suyo. Notificación lo leerá de su cola cuando pueda, así que transferencia ya no depende de que notificación esté encendido en ese instante.

2.1 El código clave

Transferencia.eventoId y @PostLoad

Cada transferencia genera su eventoId (UUID) al crearse y se guarda con la fila. Si el evento se publica dos veces lleva el mismo id, y así el consumidor puede reconocer el duplicado; si se generara al publicar, cada reintento parecería un evento nuevo. @PostLoad le asigna uno a las filas viejas de la S08 que no lo tenían.

EventoTransferencia (record)

Es el contrato con quien escucha. Si mandara la entidad, cualquier cambio de columna rompería a los consumidores; con un record propio decido exactamente qué campos salen. Está en pasado (TransferenciaRealizada) porque informa un hecho que ya ocurrió, no da una orden. También calcula su routing key.

ConfiguracionMensajeria

Declara el exchange transferencias de tipo topic y durable: topic para que cada consumidor se enlace con patrones como transferencia.*, y durable para que sobreviva a un reinicio del broker. El convertidor JSON hace que el evento viaje como JSON con el ObjectMapper de Spring, con la fecha en ISO-8601.

application.yml

publisher-confirm-type: correlated hace que el broker conteste ack o nack por cada mensaje. publisher-returns y mandatory hacen que el broker regrese el mensaje si ninguna cola lo recibió, en vez de tirarlo en silencio. connection-timeout: 2s evita esperar 60 s cuando el broker no contesta.

PublicadorDeEventos

Publica y espera la confirmación hasta 5 s. CorrelationData une la confirmación, que llega después por otro hilo, con este mensaje. Si hay nack, si se acaba el tiempo o si el mensaje regresó por mandatory, lanza EventoNoPublicadoException. La entrega PERSISTENT hace que el mensaje se escriba a disco.

Saga: guardar(t) y luego avisar

Primero guardo COMPLETADA y después publico; al revés, notificación podría avisar de una transferencia que la base todavía no registra. En compensar publico TransferenciaRevertida solo cuando el cargo ya regresó; si la devolución también falla, todavía no hay nada que avisar.

avisar()

El dinero ya se movió correctamente, así que un problema con el aviso no debe deshacer la operación ni darle error al cliente. Si publicar falla, solo queda registrado en el log y el cliente recibe su 201.

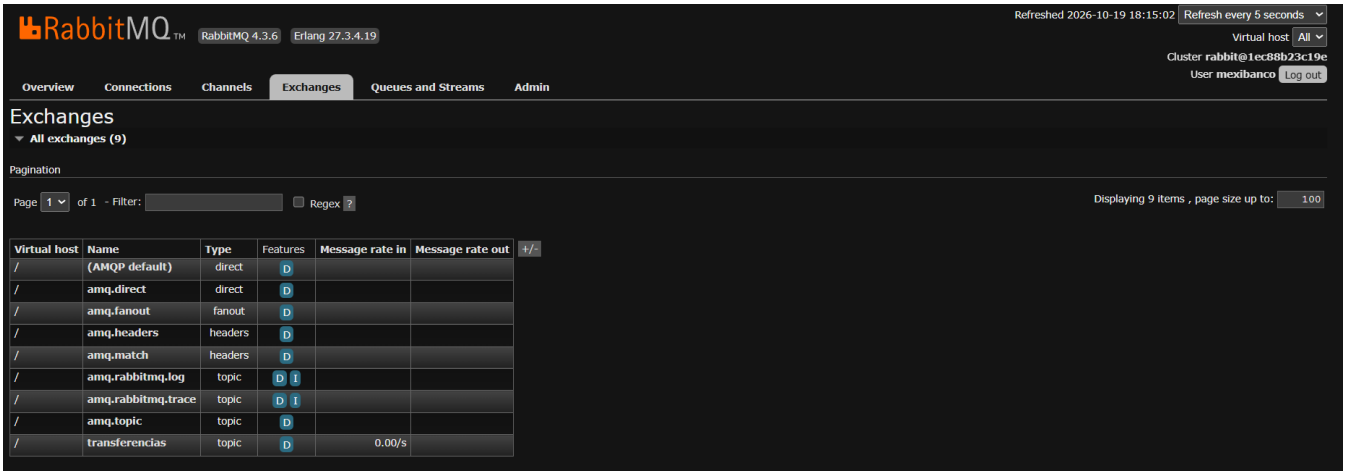
2.2 Prueba con notificación apagado

Transferencia completada con notificación apagado

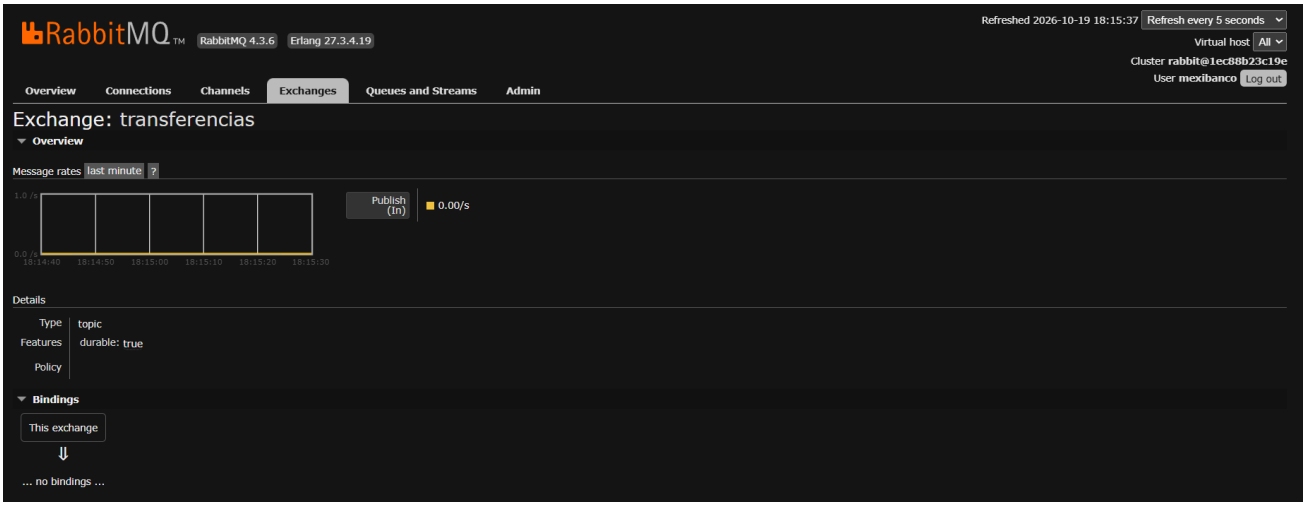
```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-S9/mexi-banco-s09 (trabajo-s09)
$ curl -i -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' \
  -d '{"claveOrigen":"0021800000000000001","claveDestino":"0021800000000000002","monto":50}'
HTTP/1.1 201 Created
transfer-encoding: chunked
Content-Type: application/json
Date: Tue, 20 Oct 2026 00:12:06 GMT

{"id":8,"claveOrigen":"0021800000000000001","claveDestino":"0021800000000000002","monto":50,"fecha":"2026-10-20T00:12:06.5785240933Z","estado
":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":900.00,"intentosDeCompensacion":0,"actualizada":"2026-10-20T00:12:06.7336630513Z","even
toId":"8b45fd5a-6cf9-4a75-8c0c-a81f83e8edda"}
```

El exchange transferencias en el panel



El exchange todavía no tiene colas enlazadas



Log de transferencia: el evento no llegó a ninguna cola

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-S9/mexi-banco-s09 (trabajo-s09)
$ docker compose logs transferencia | grep evento
transferencia-1  | 2026-10-20T00:12:06.745Z ERROR 1 --- [transferencia] [nio-8080-exec-3] m.m.transferencia.TransferenciaService   : Trans
ferencia 8: quedo terminada pero el evento TransferenciaRealizada 8b45fd5a-6cf9-4a75-8c0c-a81f83e8edda NO se publico: Ninguna cola recibio
 el evento 8b45fd5a-6cf9-4a75-8c0c-a81f83e8edda con la routing key transferencia.realizada (NO_ROUTE)
```

La transferencia respondió 201 y quedó COMPLETADA aunque notificación estaba apagado. Pero el log dice que el evento no se publicó: el exchange no tenía ninguna cola enlazada, y un exchange no guarda mensajes, solo los reparte. Gracias a mandatory, el broker regresó el mensaje en lugar de descartarlo en silencio y el publicador lo registró como no publicado. Ese aviso se perdió; se arregla cuando notificación declara su propia cola.

2.3 Qué garantiza el publisher confirm

Qué sí garantiza. Que el broker recibió el mensaje y lo guardó (a disco, por ser persistente y con cola durable) y, gracias a mandatory, que al menos una cola lo aceptó. Si algo de eso no pasa, me entero por el nack, el timeout o el mensaje regresado.

Caso 1: notificación apagado. La confirmación no garantiza que notificación procese el evento. Con notificación apagado, el broker igual confirma porque el mensaje quedó guardado en la cola; el aviso llega después, cuando el consumidor vuelve. La confirmación es entre transferencia y el broker, no con el consumidor.

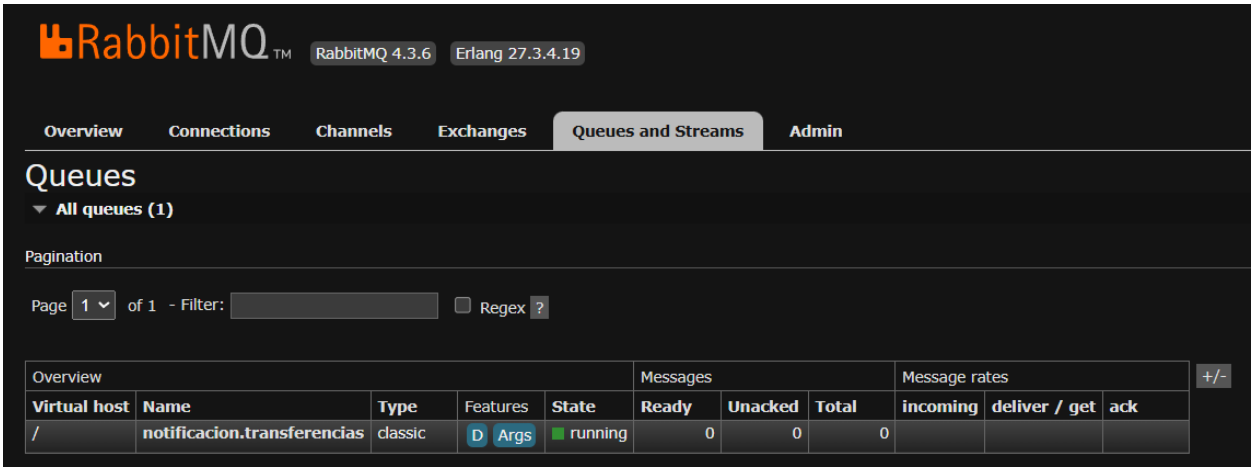
Caso 2: transferencia muere entre guardar y avisar. La confirmación no protege lo que pasa antes de publicar. Si el proceso muere después de guardar(t) y antes de avisar(), la base dice COMPLETADA pero el evento nunca se mandó y nadie lo va a reintentar. Es el problema de la doble escritura: la base y el broker no comparten una transacción.

3 Desacople en tiempo

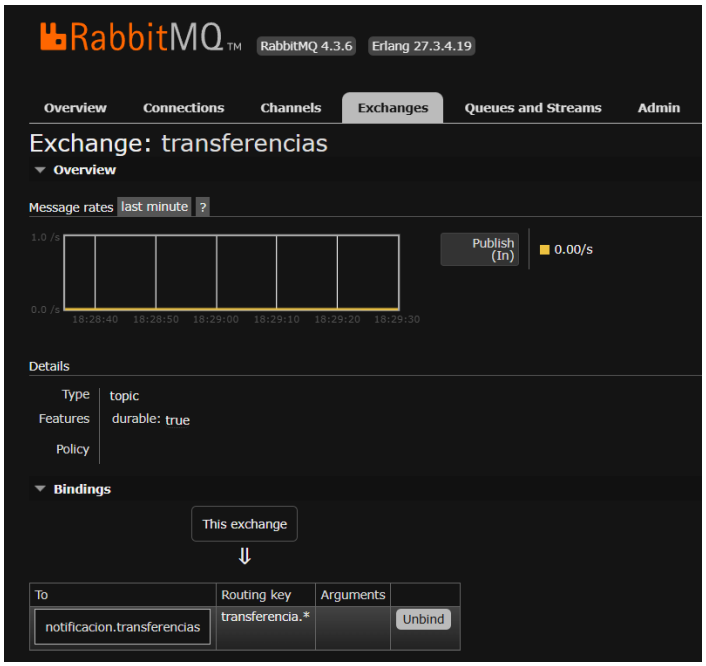
En notificación declaré la cola durable notificacion.transferencias, enlazada al exchange con el patrón transferencia.*, y un consumidor con @RabbitListener que lee el JSON, guarda los avisos en una sola transacción y solo después hace basicAck. El ack es manual (acknowledge-mode: manual) y con prefetch: 5 el broker le entrega como máximo cinco mensajes sin confirmar a la vez.

3.1 La cola de notificación

La cola notificacion.transferencias declarada por notificación



Binding del exchange hacia la cola

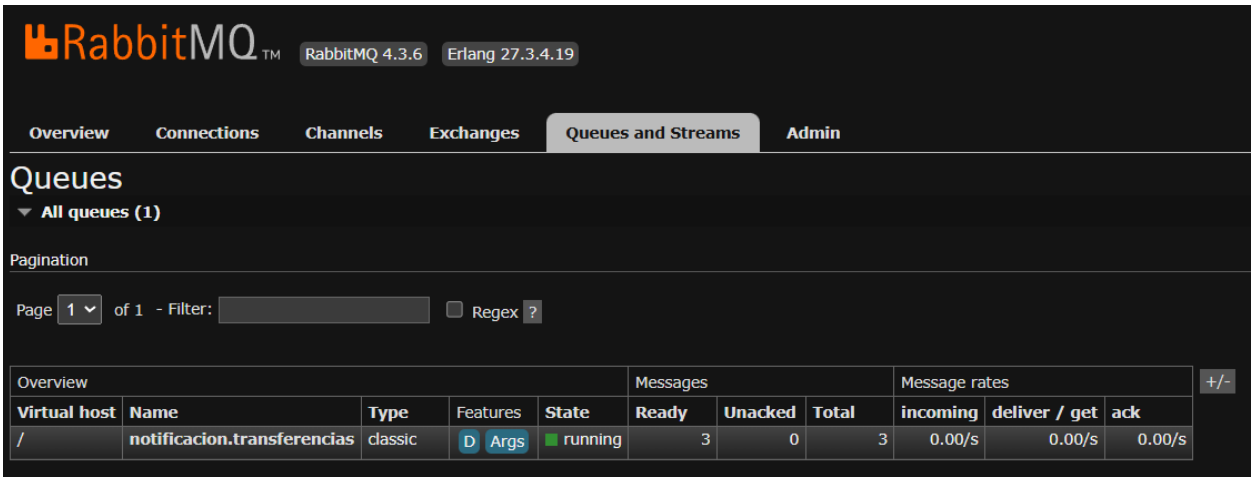


3.2 Tres transferencias con notificación apagado

Tres transferencias (11, 12 y 13) con notificación apagado

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-S9/mexi-banco-s09 (trabajo-s09)
$ for m in 11 12 13; do
> curl -s -o /dev/null -w '%{http_code}\n' -X POST http://localhost:8080/api/transferencias \
    -H 'Content-Type: application/json' \
    -d "{\"claveOrigen\":\"0021800000000000001\",\"claveDestino\":\"0021800000000000002\",\"monto\":$m}"
> done
201
201
201
```

Los eventos esperan en la cola



La cola vista desde la terminal

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-S9/mexi-banco-s09 (trabajo-s09)
$ docker compose exec rabbitmq rabbitmqctl list_queues name messages_ready messages_unacknowledged
Timeout: 60.0 seconds ...
Listing queues for vhost / ...
name    messages_ready  messages_unacknowledged
notificacion.transferencias     3       0
```

3.3 Al encender notificación

Notificación procesa los eventos al volver

```
notificacion-1  | 2026-10-20T00:31:39.693Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.m.ConsumidorDeTransferencias       : [consumidor] Transferen
ciaRealizada de la transferencia 11 (evento 1a5ff918-6417-4687-8c12-099bc059b8ef, redelivered=false): 2 aviso(s) nuevo(s)
notificacion-1  | 2026-10-20T00:39:23.702Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.m.ConsumidorDeTransferencias       : [consumidor] Transferen
ciaRealizada de la transferencia 12 (evento 0c6b8e2c-cfde-4fcc-8919-331bc79c488e, redelivered=false): 2 aviso(s) nuevo(s)
notificacion-1  | 2026-10-20T00:39:23.708Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.m.ConsumidorDeTransferencias       : [consumidor] Transferen
ciaRealizada de la transferencia 13 (evento 351f71fe-c745-4e0c-80d4-741801b38e44, redelivered=false): 2 aviso(s) nuevo(s)
```

Bandeja de Beto

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-S9/mexi-banco-s09 (trabajo-s09)
$ curl -s 'http://localhost:8080/api/notificaciones?clabe=002180000000000002'
[{"id":21,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 13, saldo resultante 636.00","fecha":"2026-10-20T00:31:
39.688959Z"},{"id":19,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 12, saldo resultante 623.00","fecha":"2026-
10-20T00:31:39.685014Z"},{"id":17,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 11, saldo resultante 611.00","f
echa":"2026-10-20T00:31:39.625479Z"}]
```

Bandeja de Ana

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-S9/mexi-banco-s09 (trabajo-s09)
$ curl -s http://localhost:8080/api/transferencias/16
{"id":16,"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":150.00,"fecha":"2026-10-20T00:45:25.814545Z","estado":"COMPENSADA"
,"motivo":"El abono a la CLABE 002180000000000002 fallo: Falla provocada con FALLAR_ABONO_A_CLABE=002180000000000002","saldoOrigenTrasCargo":678.00,"inten
tosDeCompensacion":38,"actualizada":"2026-10-20T00:51:29.859003Z","eventoId":"dca7df40-25e0-4e7e-9368-9a773cf6a480"}
```

Con notificación apagado, las tres transferencias respondieron 201 y los eventos se quedaron esperando en la cola (3 Ready y 0 consumidores). Al encender notificación, los consumió, la cola bajó a 0 y aparecieron los avisos de Ana y de Beto. Eso es el desacople en tiempo: transferencia y notificación ya no necesitan estar vivos al mismo tiempo, porque la cola guarda el mensaje mientras el consumidor no está.

3.4 Aviso de TransferenciaRevertida

Transferencia compensada con la falla provocada

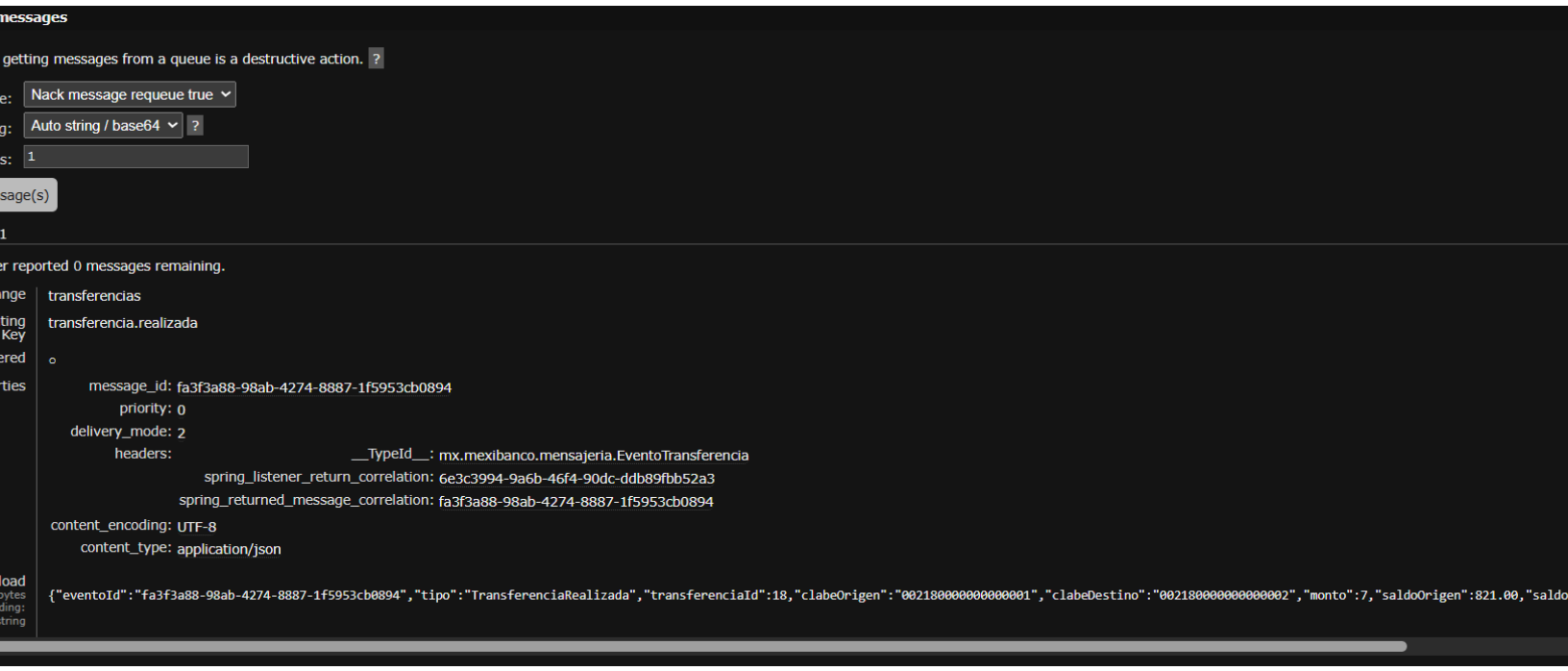
```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-S9/mexi-banco-s09 (trabajo-s09)
$ curl -i -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180000000000001","claveDestino":"002
180000000000002","monto":150}'
HTTP/1.1 502 Bad Gateway
transfer-encoding: chunked
Content-Type: application/problem+json
Date: Tue, 20 Oct 2026 00:52:24 GMT

{"type":"about:blank","title":"Transferencia no realizada","status":502,"detail":"No se pudo abonar a la CLABE 002180000000000002, asi que se devolvieron
150 a la CLABE 002180000000000001: su saldo quedo como estaba.","instance":"/transferencias","transferencia":17,"estado":"COMPENSADA","motivo":"El abono a
 la CLABE 002180000000000002 fallo: Falla provocada con FALLAR_ABONO_A_CLABE=002180000000000002"}
```

Bandeja de Ana con el aviso de devolución

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-S9/mexi-banco-s09 (trabajo-s09)
$ curl -s 'http://localhost:8080/api/notificaciones?clabe=002180000000000001'
[{"id":29,"clabeCuenta":"002180000000000001","mensaje":"DEVOLUCION_TRANSFERENCIA por 150.00, saldo resultante 828.00","fecha":"2026-10-20T00:51:29.893015Z
"},{"id":28,"clabeCuenta":"002180000000000001","mensaje":"DEVOLUCION_TRANSFERENCIA por 150.00, saldo resultante 828.00","fecha":"2026-10-20T00:51:29.87322
5Z"},{"id":26,"clabeCuenta":"002180000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 13, saldo resultante 828.00","fecha":"2026-10-20T00:39:23.709284Z"},
{"id":24,"clabeCuenta":"002180000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 12, saldo resultante 841.00","fecha":"2026-10-20T00:39:23.705466Z"},{"id"
:22,"clabeCuenta":"002180000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 11, saldo resultante 853.00","fecha":"2026-10-20T00:39:23.646733Z"},{"id":20,"
clabeCuenta":"002180000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 13, saldo resultante 864.00","fecha":"2026-10-20T00:31:39.688928Z"},{"id":18,"clabe
Cuenta":"002180000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 12, saldo resultante 877.00","fecha":"2026-10-20T00:31:39.684962Z"},{"id":16,"clabeCuent
a":"002180000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 11, saldo resultante 889.00","fecha":"2026-10-20T00:31:39.625371Z"}]
```

Con la falla provocada, el abono a Beto falló y la saga devolvió el cargo a Ana, así que llegó a COMPENSADA y publicó TransferenciaRevertida. Hay un solo aviso porque a Beto nunca le llegó dinero: la única afectada es Ana, que recibe la devolución. Hasta la S08 nadie le avisaba al cliente que su transferencia se había revertido.



4 El duplicado

RabbitMQ entrega cada mensaje al menos una vez. Si el consumidor guarda los avisos pero muere antes del ack, el broker no sabe que ya se procesó y lo vuelve a entregar con redelivered=true. También llega un duplicado si el mismo evento se publica dos veces.

4.1 Cómo lo provoqué

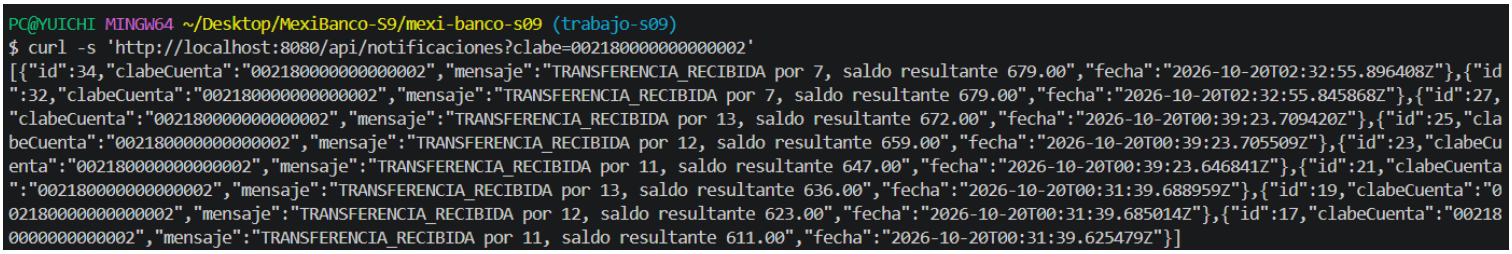
Desde el panel: Get messages con requeue y Publish message con el mismo eventoId.

Matando el consumidor durante una pausa antes del ack (PAUSA_ANTES_DEL_ACK_MS).

Apagué notificación e hice una transferencia, así que quedó un mensaje en la cola. En el panel usé Get messages con la opción de nack y requeue para ver el mensaje sin consumirlo, y copié su payload. Después publiqué ese mismo payload en el exchange transferencias con la routing key transferencia.realizada, con el mismo eventoId, y quedaron 2 mensajes Ready. Al encender notificación procesó los dos.

4.2 Antes del arreglo

Avisos repetidos en la bandeja



4.3 Mi arreglo: consumidor idempotente

Guardo cada eventoId en la tabla evento_procesado, donde es llave primaria, con un INSERT … ON CONFLICT (evento_id) DO NOTHING que regresa 1 si el evento es nuevo y 0 si ya existía. Si regresa 0, no guardo avisos y el consumidor hace ack igual, para que el mensaje salga de la cola.

La marca va en la misma transacción que los avisos: si algo falla después de marcar, el rollback se lleva la marca y los avisos juntos, y la siguiente entrega lo intenta desde cero. No lo hice como “consultar si existe y luego insertar”, porque entre esas dos llamadas otra entrega del mismo mensaje podría colarse y las dos verían que no existe. Tampoco atrapo la violación de llave, porque en Postgres eso deja la transacción abortada; ON CONFLICT evita que haya error.

4.4 Después del arreglo

La segunda entrega ya no genera avisos

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-S9/mexi-banco-s09 (trabajo-s09)
$ docker compose logs notificacion | grep consumidor
notificacion-1  | 2026-10-20T02:59:16.270Z  INFO 1 --- [notificacion] [ntContainer#
0-1] m.m.n.m.ConsumidorDeTransferencias       : [consumidor] TransferenciaRealizada
 de la transferencia 21 (evento 240f2d8c-26cf-4a7d-abe2-cfe3ab4d3bbb, redelivered=t
rue): 0 aviso(s) nuevo(s)
```

Un solo juego de avisos

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-S9/mexi-banco-s09 (trabajo-s09)
$ curl -s 'http://localhost:8080/api/notificaciones?clabe=002180000000000002'
[{"id":40,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 4, saldo resultante 697.00","fecha":"2026-10-20
T02:58:53.009689Z"},{"id":38,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 9, saldo resultante 693.00",
"fecha":"2026-10-20T02:54:56.558668Z"},{"id":36,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 5, saldo
resultante 684.00","fecha":"2026-10-20T02:47:47.777611Z"},{"id":34,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_REC
IBIDA por 7, saldo resultante 679.00","fecha":"2026-10-20T02:32:55.896408Z"},{"id":32,"clabeCuenta":"002180000000000002","mensaje"
:"TRANSFERENCIA_RECIBIDA por 7, saldo resultante 679.00","fecha":"2026-10-20T02:32:55.845868Z"},{"id":27,"clabeCuenta":"0021800000
00000002","mensaje":"TRANSFERENCIA_RECIBIDA por 13, saldo resultante 672.00","fecha":"2026-10-20T00:39:23.709420Z"},{"id":25,"clab
eCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 12, saldo resultante 659.00","fecha":"2026-10-20T00:39:23.7055
09Z"},{"id":23,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 11, saldo resultante 647.00","fecha":"2026
-10-20T00:39:23.646841Z"},{"id":21,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 13, saldo resultante 6
36.00","fecha":"2026-10-20T00:31:39.688959Z"},{"id":19,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 12
, saldo resultante 623.00","fecha":"2026-10-20T00:31:39.685014Z"},{"id":17,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERE
NCIA_RECIBIDA por 11, saldo resultante 611.00","fecha":"2026-10-20T00:31:39.625479Z"}]
```

Al repetir el experimento, la segunda entrega del mismo eventoId reporta 0 aviso(s) nuevo(s) y la bandeja tiene un solo juego de avisos.

5 La DLQ

Un mensaje envenenado es uno que nunca se va a poder procesar, por ejemplo un JSON roto. Reintentarlo no sirve porque el problema está en el mensaje mismo: falla igual cada vez y solo estorba en la cola.

5.1 Antes: el mensaje malo

Mensaje publicado desde el panel, con routing key `transferencia.realizada`: `{"hola":`

Log de notificación al recibir el mensaje malo

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-S9/mexi-banco-s09 (trabajo-s09)
$ docker compose logs notificacion | grep -E "listener failed|JsonEOFException
"
notificacion-1  | 2026-10-20T03:29:37.820Z  WARN 1 --- [notificacion] [ntConta
iner#0-1] s.a.r.l.ConditionalRejectingErrorHandler : Execution of Rabbit messa
ge listener failed.
notificacion-1  | Caused by: com.fasterxml.jackson.core.io.JsonEOFException: U
nexpected end-of-input within/between Object entries
```

Estado de la cola con el mensaje malo

Overview					Messages			Message rates			+/-
Virtual host	**Name**	**Type**	Features	**State**	**Ready**	**Unacked**	**Total**	**incoming**	**deliver / get**	**ack**	
/	**notificacion.transferencias**	classic	D Args	running	0	1	1	0.00/s	0.00/s	0.00/s	
/	**notificacion.transferencias.dlq**	classic	D Args	running	1	0	1		0.00/s	0.00/s	

El consumidor falló al leer el JSON y lanzó la excepción. Como el ack es manual, Spring no confirmó ni rechazó el mensaje, así que se quedó trabado en Unacked. No tiró el consumidor: las transferencias normales siguieron generando avisos, pero el mensaje malo ocupaba uno de los cinco lugares del prefetch y, al reiniciar notificación, se volvía a entregar y a trabar. Con varios mensajes así, el consumidor se quedaría sin lugares y dejaría de recibir.

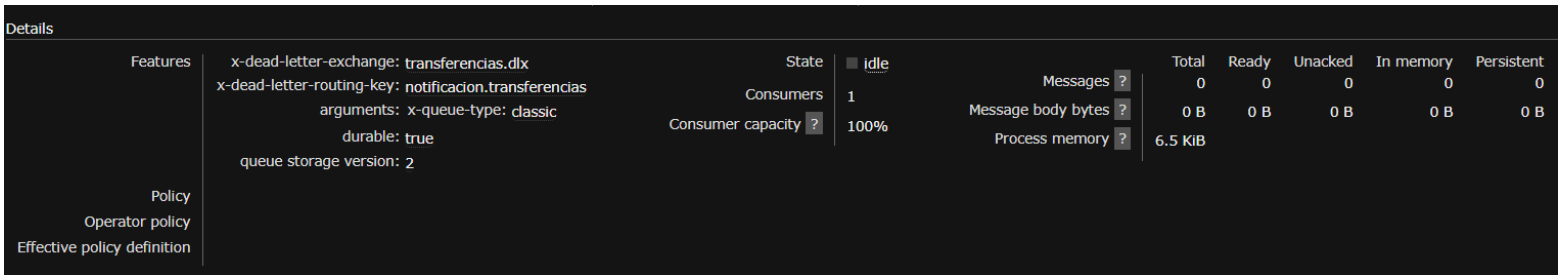
5.2 La configuración

La cola principal ahora tiene dos argumentos: x-dead-letter-exchange = transferencias.dlx y x-dead-letter-routing-key = notificacion.transferencias, que le dicen al broker a dónde mandar un mensaje rechazado. El DLX es un exchange direct y durable, y la DLQ notificacion.transferencias.dlq está enlazada a él con esa misma clave. En el consumidor, si el JSON no se puede leer o el evento no cumple el contrato, hago basicNack(etiqueta, false, false); el último false significa no reencolar, así que es el broker quien mueve el mensaje a la DLQ. Como la cola ya existía sin esos argumentos, tuve que borrarla desde el panel para que se volviera a declarar.

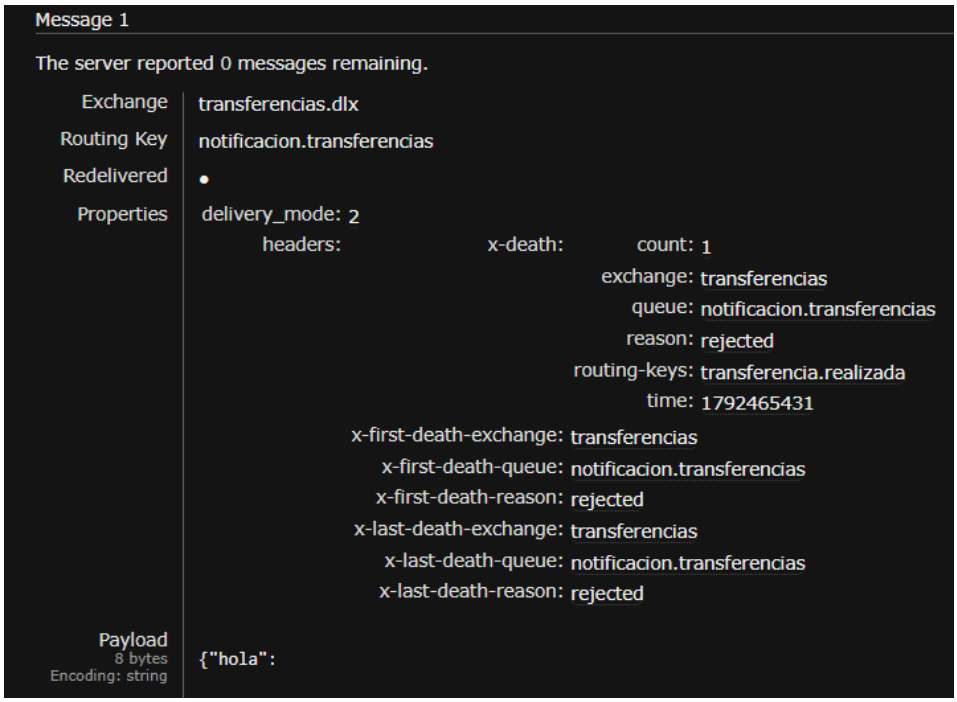
Un JSON bien formado con un `tipo` que no existe, o sin `eventoId`, también es veneno: nunca se va a poder procesar, así que lo valido y lo mando a la DLQ. En cambio, si lo que falló es mi base de datos, el mensaje está bien y el error es temporal; en ese caso hago nack con requeue para que se reintente.

5.3 Después

La cola principal con sus argumentos de dead letter



El mensaje malo en la DLQ



Ahora el mensaje malo sale de la cola principal y queda apartado en la DLQ, con la cabecera `x-death` que indica que fue rechazado, para que una persona lo revise.

6 Preguntas de cierre

6.1 RabbitMQ caído

> Pregunta Si RabbitMQ está caído cuando la saga llega a COMPLETADA, ¿qué pasa con el aviso en tu versión? ¿Por qué no lo arregla el publisher confirm?

En mi versión el aviso se pierde. La saga guarda COMPLETADA, intenta publicar y, como no hay conexión, el publicador lanza EventoNoPublicadoException después del connection-timeout de 2 s; avisar solo lo registra en el log y el cliente recibe su 201. Cuando RabbitMQ vuelve, nadie reintenta publicar ese evento. El publisher confirm no lo arregla porque solo me dice si el broker recibió el mensaje: gracias a él me entero de que falló, pero no guarda el evento en ningún lado para mandarlo después. Para eso haría falta escribir el evento en la misma transacción que el estado final, como en el patrón Outbox.

6.2 El orden del ack

> Pregunta ¿Por qué el ack va después de guardar y no antes? ¿Qué garantía obtienes con cada orden? Imagina que notificación muere justo entre las dos líneas.

Con el ack después de guardar, si notificación muere entre las dos líneas, los avisos ya están en la base pero el broker no recibió el ack, así que vuelve a entregar el mensaje: el aviso puede repetirse, pero nunca se pierde (entrega al menos una vez). Con el ack antes de guardar, si muere en medio, el broker ya borró el mensaje y los avisos nunca se guardaron: el aviso se pierde, aunque nunca se duplica (a lo más una vez). En un banco prefiero no perder avisos, así que el ack va después y los duplicados los resuelvo con la idempotencia por eventoId.

6.3 Elige y di qué sacrificaste

> Pregunta Al quitar la llamada síncrona ganaste que el aviso no tumbe la transferencia. ¿Qué perdiste?

Elección: Desacoplar con mensajería asíncrona: transferencia publica el evento y no espera a notificación.

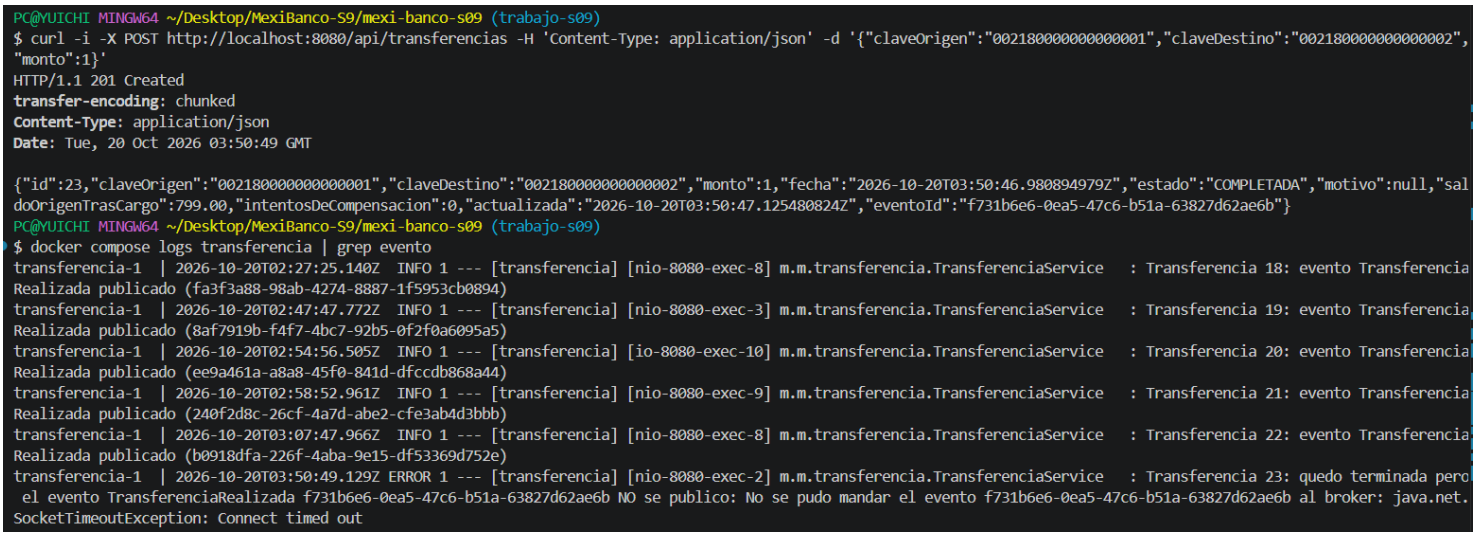
Porque: Un aviso puede esperar, pero una transferencia que ya movió el dinero no debe fallar porque el servicio de avisos esté caído.

Sacrifico: La certeza inmediata: cuando el cliente recibe su 201 no sé si el aviso ya llegó o llegará después (consistencia eventual), y además sumo un broker que operar, duplicados que manejar y fallas más difíciles de rastrear.

7 Reto opcional: Outbox

7.1 El problema: doble escritura

Con RabbitMQ apagado el evento se pierde



La saga escribe en dos sistemas: guarda COMPLETADA en Postgres y después publica el evento en RabbitMQ. No hay una transacción que abarque a los dos. Con el broker apagado, la base quedó en COMPLETADA y el cliente recibió su 201, pero la publicación falló después del connection-timeout y el evento solo quedó en el log. Cuando RabbitMQ volvió, nadie lo reintentó y el aviso nunca llegó. Pasaría lo mismo si transferencia muriera entre guardar y publicar. Es el problema de la doble escritura: base y broker son dos sistemas sin una transacción que los una.

7.2 Mi diseño

La saga ya no publica. Guarda el estado final y el evento en la misma transacción local, en una tabla outbox de la base de transferencia, así que los dos quedan escritos o ninguno. Uso un TransactionTemplate porque @Transactional en un método privado, o llamado desde la misma clase, no hace nada. Un relay @Scheduled corre cada segundo: toma las filas pendientes, las publica con PublicadorDeEventos, que espera la confirmación del broker, y las marca como publicadas. Si el broker no confirma, la fila sigue pendiente con un intento más y se reintenta en la siguiente vuelta. La consulta usa FOR UPDATE SKIP LOCKED: si hay varias réplicas de transferencia, cada relay bloquea sus filas y se salta las que ya tomó otro, así no publican el mismo evento dos veces.

7.3 Prueba

El evento queda pendiente en la outbox

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-S9/mexi-banco-s09 (trabajo-s09)
$ curl -i -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"00218000000000001","claveDestino":"00218000000000002",
"monto":2}'
HTTP/1.1 201 Created
transfer-encoding: chunked
Content-Type: application/json
Date: Tue, 20 Oct 2026 04:01:33 GMT

{"id":24,"claveOrigen":"00218000000000001","claveDestino":"00218000000000002","monto":2,"fecha":"2026-10-20T04:01:34.421597597Z","estado":"COMPLETADA","motivo":null,"sal
doOrigenTrasCargo":797.00,"intentosDeCompensacion":0,"actualizada":"2026-10-20T04:01:34.495732144Z","eventoId":"cee87330-03e9-4ed6-a9eb-a84b4165e5c0"}
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-S9/mexi-banco-s09 (trabajo-s09)
$ docker compose exec db psql -U mexibanco -d transferencia -c "SELECT evento_id, tipo, transferencia_id, creado, publicado, intentos FROM outbox ORDER BY creado DESC LIMI
T 3;"
             evento_id              |         tipo          | transferencia_id |            creado             | publicado | intentos
--------------------------------------+-----------------------+------------------+-------------------------------+-----------+----------
 cee87330-03e9-4ed6-a9eb-a84b4165e5c0 | TransferenciaRealizada |               24 | 2026-10-20 04:01:34.501133+00 |           |        9
(1 row)
```

El relay publica al volver el broker

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-S9/mexi-banco-s09 (trabajo-s09)
$ docker compose logs transferencia | grep relay
transferencia-1  | 2026-10-20T04:01:39.171Z  WARN 1 --- [transferencia] [   scheduling-1] mx.mexibanco.mensajeria.RelayDeOutbox    : [relay] evento TransferenciaRealizada
cee87330-03e9-4ed6-a9eb-a84b4165e5c0 de la transferencia 24 sigue pendiente (intento 1): No se pudo mandar el evento cee87330-03e9-4ed6-a9eb-a84b4165e5c0 al broker: java.n
et.UnknownHostException: rabbitmq: Name does not resolve
transferencia-1  | 2026-10-20T04:01:40.188Z  WARN 1 --- [transferencia] [   scheduling-1] mx.mexibanco.mensajeria.RelayDeOutbox    : [relay] evento TransferenciaRealizada
cee87330-03e9-4ed6-a9eb-a84b4165e5c0 de la transferencia 24 sigue pendiente (intento 2): No se pudo mandar el evento cee87330-03e9-4ed6-a9eb-a84b4165e5c0 al broker: java.n
et.UnknownHostException: rabbitmq
transferencia-1  | 2026-10-20T04:01:41.204Z  WARN 1 --- [transferencia] [   scheduling-1] mx.mexibanco.mensajeria.RelayDeOutbox    : [relay] evento TransferenciaRealizada
cee87330-03e9-4ed6-a9eb-a84b4165e5c0 de la transferencia 24 sigue pendiente (intento 3): No se pudo mandar el evento cee87330-03e9-4ed6-a9eb-a84b4165e5c0 al broker: java.n
et.UnknownHostException: rabbitmq
transferencia-1  | 2026-10-20T04:01:42.208Z  WARN 1 --- [transferencia] [   scheduling-1] mx.mexibanco.mensajeria.RelayDeOutbox    : [relay] evento TransferenciaRealizada
cee87330-03e9-4ed6-a9eb-a84b4165e5c0 de la transferencia 24 sigue pendiente (intento 4): No se pudo mandar el evento cee87330-03e9-4ed6-a9eb-a84b4165e5c0 al broker: java.n
et.UnknownHostException: rabbitmq
transferencia-1  | 2026-10-20T04:01:43.224Z  WARN 1 --- [transferencia] [   scheduling-1] mx.mexibanco.mensajeria.RelayDeOutbox    : [relay] evento TransferenciaRealizada
cee87330-03e9-4ed6-a9eb-a84b4165e5c0 de la transferencia 24 sigue pendiente (intento 5): No se pudo mandar el evento cee87330-03e9-4ed6-a9eb-a84b4165e5c0 al broker: java.n
et.UnknownHostException: rabbitmq
transferencia-1  | 2026-10-20T04:01:44.240Z  WARN 1 --- [transferencia] [   scheduling-1] mx.mexibanco.mensajeria.RelayDeOutbox    : [relay] evento TransferenciaRealizada
cee87330-03e9-4ed6-a9eb-a84b4165e5c0 de la transferencia 24 sigue pendiente (intento 6): No se pudo mandar el evento cee87330-03e9-4ed6-a9eb-a84b4165e5c0 al broker: java.n
et.UnknownHostException: rabbitmq
transferencia-1  | 2026-10-20T04:01:45.255Z  WARN 1 --- [transferencia] [   scheduling-1] mx.mexibanco.mensajeria.RelayDeOutbox    : [relay] evento TransferenciaRealizada
cee87330-03e9-4ed6-a9eb-a84b4165e5c0 de la transferencia 24 sigue pendiente (intento 7): No se pudo mandar el evento cee87330-03e9-4ed6-a9eb-a84b4165e5c0 al broker: java.n
et.UnknownHostException: rabbitmq
transferencia-1  | 2026-10-20T04:01:46.259Z  WARN 1 --- [transferencia] [   scheduling-1] mx.mexibanco.mensajeria.RelayDeOutbox    : [relay] evento TransferenciaRealizada
cee87330-03e9-4ed6-a9eb-a84b4165e5c0 de la transferencia 24 sigue pendiente (intento 8): No se pudo mandar el evento cee87330-03e9-4ed6-a9eb-a84b4165e5c0 al broker: java.n
et.UnknownHostException: rabbitmq
transferencia-1  | 2026-10-20T04:01:47.276Z  WARN 1 --- [transferencia] [   scheduling-1] mx.mexibanco.mensajeria.RelayDeOutbox    : [relay] evento TransferenciaRealizada
cee87330-03e9-4ed6-a9eb-a84b4165e5c0 de la transferencia 24 sigue pendiente (intento 9): No se pudo mandar el evento cee87330-03e9-4ed6-a9eb-a84b4165e5c0 al broker: java.n
et.UnknownHostException: rabbitmq
```

El aviso llega

```
PC@YUICHI MINGW64 ~/Desktop/MexiBanco-S9/mexi-banco-s09 (trabajo-s09)
$ curl -s 'http://localhost:8080/api/notificaciones?clabe=00218000000000002'
[{"id":44,"clabeCuenta":"00218000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 2, saldo resultante 703.00","fecha":"2026-10-20T04:02:17.609667Z"}
```

> Pregunta Si el relay publica y muere antes de marcar la fila, el evento sale otra vez con el mismo eventoId. ¿Quién te salva de ese duplicado? Y con el broker apagado, ¿transferencia sigue healthy?

Del duplicado me salva notificación. Cada evento sale con el mismo eventoId y notificación lo marca en evento_procesado con INSERT … ON CONFLICT DO NOTHING, en la misma transacción que los avisos. Si el relay publica y muere antes de marcar la fila, el evento sale otra vez, la segunda entrega encuentra la marca y reporta 0 avisos nuevos. El outbox garantiza al menos una vez y la idempotencia del consumidor hace que el efecto sea uno solo.

Sobre la salud: antes del cambio, no seguía healthy. Spring Boot agrega solo el indicador de RabbitMQ a /actuator/health, así que con el broker apagado el estado pasaba a DOWN y el healthcheck de Compose marcaba a transferencia como unhealthy aunque atendiera bien. Con el outbox, transferencia ya no necesita al broker para completar una transferencia, así que desactivé ese indicador (management.health.rabbit.enabled: false). Ahora sigue healthy y los eventos se acumulan en el outbox hasta que RabbitMQ vuelve.